CSC 374 Programmation Systèmes II

TP1 : Cache Lab, simuler un cache et l'exploiter

iPNet, filières Réseaux et télécommunications, Systèmes et sécurité, Cybersécurité, Cloud et Développement logiciel. Énoncé du travail pratique 1, lancé en séance 2, à remettre au début de la séance 4.

Présentation

Objectifs

Ce travail pratique est adapté du *Cache Lab* que Bryant et O'Hallaron distribuent avec leur ouvrage [CSAPP]. Il prolonge les séances 2 et 3 et en vérifie la maîtrise sur deux plans.

- **Partie A, comprendre.** Écrire en C un simulateur de cache, `csim`, qui rejoue une trace d'accès mémoire produite par `valgrind` et compte les succès, les défauts et les évictions d'un cache de paramètres quelconques, avec remplacement LRU.
- **Partie B, exploiter.** Écrire une fonction de transposition de matrice qui fait le moins de défauts possible dans un petit cache à correspondance directe.

À l'issue du TP, l'étudiant doit être capable de traduire le modèle d'un cache en structures de données C, d'écrire un programme système robuste (options, fichiers, erreurs, mémoire dynamique), de prévoir les défauts d'un parcours et de transformer un algorithme pour qu'il exploite la localité. Ces compétences relèvent du résultat d'apprentissage RA1 du syllabus.

Calendrier et modalités

Tableau 1. Calendrier du TP1.

Étape	Date	Contenu
Lancement	séance 2	présentation (manuscrit de la séance 2, partie 7), distribution du dossier
Partie A	avant la séance 3	tout le nécessaire a été vu en séance 2
Partie B	après la séance 3	la technique du blocage est présentée en séance 3
Remise	début de la séance 4	archive produite par `make remise`, déposée sur la plateforme du cours

Le travail est individuel. Les échanges d'idées entre étudiants sont permis ; l'échange de code ne l'est pas. Les outils d'aide à la programmation, y compris les assistants fondés sur l'IA, sont autorisés, mais l'étudiant reste l'auteur de ce qu'il remet : l'enseignant peut lui demander d'expliquer oralement n'importe quelle ligne de son code ou de son rapport, et une explication défaillante annule les points correspondants. Tout retard est pénalisé de 2 points par jour entamé.

Environnement

Le TP se fait sur la machine Linux du cours (Ubuntu 24.04, `gcc` 13, `valgrind` 3.22, Python 3). Les outils de test utilisent `valgrind` pour la partie B : ils ne fonctionnent pas sous macOS ni sous Windows sans WSL.

Le dossier fourni

Tableau 2. Contenu du dossier `CSC374_TP1_code_etudiant`.

Fichier	Rôle	À modifier ?
`csim.c`	squelette du simulateur (partie A)	oui

Fichier	Rôle	À modifier ?
`trans.c`	fonction de transposition, version naïve (partie B)	oui
`cachelab.h`, `cachelab.c`	fonctions fournies : `afficher_bilan`, `est_transposee`	non
`tracegen.c`	banc d'essai de la partie B	non
`csim-ref`	simulateur de référence (exécutable)	non
`test-csim.py`	évaluation automatique de la partie A	non
`test-trans.py`	évaluation automatique de la partie B	non
`traces/`	sept traces au format de Lackey	non
`outils/simcache.py`	simulateur Python du cours (séance 2), utilisable comme second oracle	non
`Makefile`	cibles `csim`, `tracegen`, `test`, `fuites`, `remise`, `clean`	non

Pour commencer :

```
$ cd CSC374_TP1_code_etudiant
$ chmod +x csim-ref           # si le dossier a transité par Windows
$ make csim
$ python3 test-csim.py        # 0 / 8 avec le squelette
$ python3 test-trans.py       # 0 / 9 avec la version naïve
```

Partie A : le simulateur de cache (8 points)

Spécification

Le programme `csim` s'utilise ainsi :

```
Usage : ./csim [-hv] -s <s> -E <E> -b <b> -t <fichier_trace>
  -h          affiche l'aide
  -v          mode bavard : affiche le résultat de chaque accès
  -s <s>      nombre de bits d'indice d'ensemble (S = 2^s ensembles)
  -E <E>      associativité (nombre de lignes par ensemble)
  -b <b>      nombre de bits de décalage (B = 2^b octets par bloc)
  -t <trace>  fichier de trace au format de Lackey
```

Il simule un cache de $S = 2^s$ ensembles de E lignes, avec des blocs de $B = 2^b$ octets et un remplacement LRU exact, sur la trace donnée, puis appelle une seule fois la fonction fournie `afficher_bilan(succes, defauts, evictions)`. Cette fonction écrit le bilan sur la sortie standard et dans le fichier `.csim_resultats`, que lit l'outil de test :

```
$ ./csim -s 1 -E 2 -b 2 -t traces/exercice2.trace
hits:4 misses:5 evictions:2
```

Format des traces et règles de simulation

Chaque ligne d'une trace a la forme `[espace]opération adresse,taille` (manuscrit de la séance 2, partie 7.2) :

```
I  001090e1,7
 L 0010c060,4
 M 0010dff8,4
 S 1ffeffa27c,4
```

1. Les lignes qui ne commencent pas par une espace (lignes `I` des instructions) sont ignorées.
2. `L` (lecture) et `S` (écriture) comptent pour un accès ; `M` (modification) compte pour deux accès à la même adresse, une lecture puis une écriture.
3. La taille est ignorée : on suppose que chaque accès tient dans un seul bloc.
4. Les adresses sont des entiers de 64 bits écrits en hexadécimal, sans préfixe `0x`.
5. Un **succès** est un accès dont le bloc est dans le cache ; un **défaut**, un accès dont le bloc n'y est pas ; une **éviction**, un défaut qui retire une ligne valide pour faire place au nouveau bloc. Lectures et écritures sont traitées de la même façon.

Exigences

- **Paramètres quelconques.** La structure du cache est allouée dynamiquement selon s, E et b, et libérée à la fin. Aucune taille maximale ne doit être codée en dur. `make fuites` exécute `csim` sous `valgrind --leak-check=full` : aucune fuite ni erreur ne doit être signalée.
- **Options.** Les options sont analysées avec `getopt` [`getopt(3)`]. Une option obligatoire absente, une valeur non numérique ou hors domaine ($E \geq 1$; $s + b < 64$) provoque un message sur la sortie d'erreur, l'affichage de l'aide et un code de retour non nul. L'option `-h` affiche l'aide et termine avec le code 0.
- **Fichiers.** Un fichier de trace introuvable ou illisible, ou une ligne mal formée, provoquent un message d'erreur qui nomme le fichier (et la ligne), et un code de retour non nul.
- **Mode bavard.** Avec `-v`, `csim` affiche, pour chaque opération de la trace, l'opération suivie des résultats de ses accès :

```
L 30,1 miss eviction
M 0010dff8,4 miss hit
```

- **Qualité.** Le programme compile sans avertissement avec `-std=c17 -Wall -Wextra -Wpedantic`. Les fonctions sont courtes et commentées ; l'en-tête du fichier décrit la représentation du cache et la réalisation de LRU.

Évaluation automatique

`test-csim.py` compile `csim` puis compare son bilan à celui de `csim-ref` sur huit cas, à 1 point chacun, accordé seulement si les trois nombres sont identiques :

Tableau 3. Les huit cas de test de la partie A.

Trace	s	E	b	**Ce que le cas vérifie**
`exercice2.trace`	1	2	2	exercice résolu 2 de la séance 2
`entr4.trace`	2	1	3	exercice d'entraînement 4, correspondance directe
`cyclique_13.trace`	6	12	6	LRU sur un cycle de 13 lignes
`prodscal_0.trace`	6	1	6	conflits du produit scalaire
`ordre_colonnes_64.trace`	4	4	6	associativité, parcours par colonnes
`trans_32x32.trace`	0	32	5	cache entièrement associatif ($s = 0$)
`histo.trace`	4	2	4	lignes `I` et `M`, adresses de pile sur 37 bits
`histo.trace`	8	4	6	grand cache

Conseils

1. Écrire d'abord l'analyse des options et la lecture de la trace, en affichant ce qui est lu ; vérifier sur `exercice2.trace` et sur `histo.trace`.
2. Écrire la décomposition d'adresse et la vérifier à la main sur les exemples de la séance 2 (exercice résolu 1, exercice d'entraînement 10).
3. Réaliser d'abord la correspondance directe ($E = 1$), où il n'y a pas de victime à choisir, puis l'associativité avec LRU. Un compteur global incrémenté à chaque accès, dont on mémorise la valeur dans la ligne à chaque usage, suffit pour LRU.
4. Comparer avec `-v` ligne à ligne sur les petites traces, puis avec `./csim-ref` et `python3 outils/simcache.py -s ... -E ... -b ... trace` sur les grandes.

Pièges connus : lire une adresse de 64 bits dans un `int` ; calculer `1 << s` en `int` ; oublier que `M` produit deux accès ; compter une éviction quand l'ensemble avait encore une ligne libre ; ne pas libérer la mémoire sur les chemins d'erreur.

Partie B : la transposition (9 points)

Spécification

Il s'agit de compléter, dans `trans.c`, la fonction

```
void transposer_soumission(int M, int N, int A[N][M], int B[M][N]);
```

qui range dans `B` la transposée de `A` : `B[j][i] == A[i][j]` pour tout $0 \leq i < N$ et $0 \leq j < M$. La fonction est évaluée dans un cache à correspondance directe de paramètres $s = 5$, $E = 1$, $b = 5$ (32 ensembles d'une ligne de 32 octets, 1 Kio au total), pour trois tailles : 32 × 32, 64 × 64 et 61 × 67 (`M = 61` colonnes, `N = 67` lignes). On peut écrire des fonctions auxiliaires et choisir la méthode selon `M` et `N`.

Règles

Les règles suivantes sont vérifiées à la lecture du code. Une règle enfreinte annule les points de la taille concernée.

1. Au plus 12 variables locales de type `int` à la fois, paramètres non compris : si une fonction en appelle une autre, les variables des deux s'additionnent.
2. Aucun tableau local, aucune variable globale ou statique, aucune allocation dynamique : les seules données en mémoire sont `A` et `B`.
3. Pas de récursivité.
4. `A` n'est jamais modifié ; `B` peut servir de mémoire provisoire.

Comment la fonction est évaluée

`test-trans.py` compile `tracegen.c` avec `trans.c`, sans optimisation (`-O0`). Le banc d'essai place `A` et `B` dans deux tableaux statiques de 256 × 256 `int`, alignés, `B` commençant 256 Kio après `A` : `A[i][j]` et `B[i][j]` tombent donc dans le même ensemble du cache. Il remplit `A`, puis appelle `transposer_soumission` entre deux écritures dans une variable témoin. L'outil exécute ce programme sous Lackey, ne garde que les accès aux tableaux `A` et `B` faits entre les deux écritures témoins, les rejoue dans `csim-ref -s 5 -E 1 -b 5`, et vérifie que la transposition est correcte. Les accès aux variables locales ne sont pas comptés : c'est pourquoi la règle 2 interdit de ranger des données ailleurs que dans `A` et `B`.

Tableau 4. Barème de la partie B. Entre les deux seuils, les points décroissent linéairement. Une transposition incorrecte vaut 0.

Taille	Défauts de la version naïve	Totalité des points si les défauts sont au plus	Aucun point à partir de	Points
32 × 32	1 180	300	600	3
64 × 64	4 720	1 300	2 000	3
61 × 67	4 420	2 000	3 000	3

```
$ python3 test-trans.py
  Taille  Correct   Accès  Succès  Défauts      Seuils  Points
 32 x 32       oui    2048     868     1180   300-600    0.00/3
 64 x 64       oui    8192    3472     4720  1300-2000   0.00/3
 61 x 67       oui    8174    3754     4420  2000-3000   0.00/3
Total partie B : 0.00 / 9
```

L'option `-v` conserve les traces filtrées (`trans_32x32.trace`, etc.), qu'on peut analyser avec `python3 outils/simcache.py -s 5 -E 1 -b 5 --3c trans_32x32.trace` pour connaître la nature des défauts.

Conseils

1. Commencer par 32 × 32, dont la partie 7.4 du manuscrit de la séance 2 analyse la version naïve au défaut près. Calculer le nombre minimal de défauts (les défauts obligatoires) : c'est l'objectif à approcher.
2. Appliquer le blocage de la séance 3 : transposer par sous-matrices assez petites pour que les blocs de `A` et de `B` qu'elles touchent tiennent ensemble dans le cache. Choisir la taille des sous-matrices en fonction de la taille d'un bloc de cache et de la distance, en ensembles, entre deux lignes consécutives de la matrice.
3. Examiner ce qui se passe sur la diagonale, où `A` et `B` se disputent les mêmes ensembles. Les variables locales permettent de lire plusieurs éléments de `A` avant d'écrire dans `B`.
4. Pour 64 × 64, la méthode qui convient à 32 × 32 ne suffit pas : calculer quelles lignes de `B` se disputent le même ensemble. `B` peut servir de mémoire provisoire.
5. Pour 61 × 67, les dimensions ne sont pas des puissances de deux : les conflits sont irréguliers. Mesurer plusieurs tailles de blocs plutôt que calculer.
6. Mesurer à chaque modification et noter les résultats : ils serviront au rapport.

Rapport et qualité (3 points)

Le rapport, au format PDF, deux pages au plus, s'appelle `rapport.pdf` et se trouve dans le dossier du TP. Il contient :

1. pour la partie A, la représentation du cache, la réalisation de LRU et le traitement des erreurs (1 point) ;
2. pour la partie B, la stratégie retenue pour chaque taille, justifiée par un raisonnement sur les ensembles du cache du même type que celui de la partie 7.4 du manuscrit, et le tableau des défauts mesurés pour chaque étape essayée (1 point) ;
3. la qualité du code est notée sur le dernier point : compilation sans avertissement, absence de fuite, messages d'erreur, lisibilité, commentaires.

Un nombre de défauts annoncé sans explication ne vaut que la moitié des points de la ligne 2.

Remise

La commande `make remise` produit, dans le dossier parent, l'archive `<identifiant>_tp1.tar.gz` qui contient `csim.c`, `trans.c` et `rapport.pdf`. Seule cette archive est déposée. Avant de la déposer, vérifier dans une copie propre du dossier fourni que `make test` donne les résultats attendus avec les fichiers remis : l'enseignant évaluera exactement de cette manière.

Tableau 5. Grille d'évaluation. La note sur 20 compte pour 6 % de la note finale.

Élément	Points
Partie A : 8 cas de test	8
Partie B : 32 × 32, 64 × 64, 61 × 67	9
Rapport : représentation et LRU	1
Rapport : stratégies justifiées et mesures	1
Qualité du code	1
Total	20

Références

- [CSAPP] R. E. Bryant, D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3e édition, Pearson, 2016. §6.2 à §6.6 ; problème 6.45 (transposition).

- R. E. Bryant, D. R. O'Hallaron, *Cache Lab*, Carnegie Mellon University, dont ce TP est adapté.
- Manuscrits des séances 2 et 3 du cours.
- Manuel en ligne de Linux : `getopt(3)`, `fopen(3)`, `fgets(3)`, `sscanf(3)`, `strtol(3)`, `malloc(3)` ; manuel de valgrind, chapitre « Lackey ».